AI PROMPT DOCUMENTATION

ApplyTrack — Job Application Tracker

Technology stack: Python, Flask, SQLite, HTML, CSS

1. Purpose of this document

This document records the main AI-assisted development instructions and follow-up refinements associated with ApplyTrack. It is a reconstructed summary, not a verbatim export of every message in the original chat. The initial project structure and code were generated by the AI assistant from the assessment context; the user did not independently write all the prompts listed below. The prompts are representative development instructions, not claims that each sentence was typed exactly as shown.

2. Initial project-planning instruction

Representative prompt / instruction:

"Create a beginner-friendly technology project suitable for an AI-assisted development assessment. Choose a practical problem and build a usable web application with Python. Include source files, setup instructions, and documentation explaining the project."

Purpose: Establish the project direction, choose a manageable scope, and produce a project that can be demonstrated and explained.

3. Project selection and feature definition

Representative prompt / instruction:

"Build ApplyTrack, a job application tracker using Flask and SQLite. Let users add an application with company name, job role, application date, status, and notes. Show saved applications and provide ways to update status, search or filter records, and delete an entry."

Purpose: Turn the idea into concrete features and select a lightweight database.

4. Backend and database implementation

Representative prompt / instruction:

"Create the Flask backend and SQLite database logic for the tracker. Include routes to display applications, add a new record, update its status, and delete a record. Keep the code understandable for a beginner and include the required database table setup."

Purpose: Generate the application logic and persistence layer.

5. Frontend and styling

Representative prompt / instruction:

“Create the HTML template and CSS for a clean dashboard. Show summary counts for total applications, currently applied applications, and follow-ups needed. Include an application form, status labels, and a readable list of saved applications. Link the stylesheet correctly from the Flask template.”

Purpose: Provide the interface and styling, with HTML templates and static CSS in Flask's expected folders.

6. Follow-up refinement: files and folder structure

Representative prompt / instruction:

“Make sure all required files are included and explain where index.html belongs. Show the exact folder structure, including templates/index.html and static/style.css, so Flask can find the template and stylesheet.”

Why it mattered: Flask expects templates in a templates folder by default and static assets in a static folder. Clarifying this prevents missing-template and missing-CSS problems.

7. Follow-up refinement: CSS not appearing

Representative prompt / instruction:

“If the page appears without the expected colours or CSS, check that the stylesheet exists in the static folder and that the HTML contains the correct Flask stylesheet link: url_for('static', filename='style.css'). Explain how to verify the fix in the browser.”

Purpose: Troubleshoot the connection between the HTML template and stylesheet.

8. Documentation and assessment deliverables

Representative prompt / instruction:

“Prepare project documentation explaining the purpose, features, technology stack, setup steps, and how to run the app. Also prepare prompt-process documentation and a short video presentation script describing the AI-assisted approach, decisions, challenges, and intended outcome.”

Purpose: Make the project easier to run, demonstrate, and explain for assessment.

9. Significant iteration summary

Stage	Issue / need	Refinement
Planning	A suitable assessment project was needed.	Choose a practical productivity tool with a manageable scope.
Feature design	The idea needed to become a usable app.	Define add, view, update, delete, search/filter, and follow-up features.
Interface	The app needed a clear dashboard and styling.	Create HTML and CSS, summary counts, and status labels.
File structure	The HTML template was not easy to locate.	Clarify that index.html belongs in templates/ and CSS in static/.
Styling	Colours might not appear if CSS is not linked.	Check the static file path and Flask url_for stylesheet reference.
Submission	The assessment requires	Prepare setup instructions, prompt

	explanation and documentation.	documentation, and a presentation script.

10. AI tools and approach

AI assistant: ChatGPT. The project was generated with AI assistance. The intended stack uses Flask for the web application, SQLite for storing application records, HTML for page structure, and CSS for presentation. The user should run the project locally, inspect the code, and verify each feature before describing it as tested or complete.

11. Testing and limitations

Run the app on your own computer before submission. Confirm that it starts, the page loads, CSS appears, records can be added and updated, and the follow-up indicator behaves as intended. Do not claim a test passed unless you actually performed it. The project is a local demo unless separately deployed; http://127.0.0.1:5000 is not a public project link.

12. Reflection

AI-generated code still needs human review and testing. Clear requirements, small feature groups, and specific follow-up questions can make results easier to improve. Before presenting the project, be ready to explain the purpose of each file, how Flask handles requests, how SQLite stores records, and how the HTML template connects to the CSS stylesheet.

Important honesty note: This is a reconstructed prompt log based on the project workflow, not an exact transcript of prompts personally typed by the user. Update the testing section after actually running the project.